# Optional Assignment

December 30, 2025

## 1 Introduction

This report presents the implementation of two deep learning models that transform RGB images of size 3×32×32 into grayscale images of size 1×28×28, with horizontal and vertical flips applied. Both models are trained on the CIFAR-10 dataset. The first model (Model 1) is optimized for inference speed, achieving faster performance than sequential CPU transformations, while the second model (Model 2) prioritizes prediction quality, achieving significantly lower training and validation losses with predictions very close to the ground truth.

## 2 Model Architectures

### 2.1 Model 1:

Model 1 uses a simple convolutional architecture designed for efficiency and speed:

- 4 convolutional layers with ReLU activations
- Downsampling via stride=2 in the second layer
- Dilated convolutions to increase receptive field
- Final 1×1 convolution for RGB to grayscale conversion

The proposed architecture is deliberately designed to be simple and efficient, enabling fast training and inference while maintaining good performance. Its compact structure allows quick execution on CPU, making it suitable for real-time applications. Early downsampling reduces spatial dimensions and computational cost in later layers, while the use of dilated

convolutions increases the receptive field without adding extra parameters, enabling the model to capture broader contextual information efficiently. The network is trained in an end-to-end manner, learning the full transformation directly, and the final upsampling step relies on nearest neighbor interpolation, which is both fast and sufficient for the required resizing operation.

### 2.2 Model 2

Model 2 uses a U-Net inspired encoder-decoder architecture designed for high prediction quality:

- Encoder with two downsampling blocks and a bottleneck
- Decoder with skip connections to preserve fine details
- Each block uses dilated convolutions for increased

Each ConvBlock consists of two 3×3 convolutions with ReLU activations, where the second convolution uses dilation=2 to increase receptive field.

The U-Net–inspired architecture is designed with a strong emphasis on output quality. Its encoder–decoder structure with skip connections preserves fine-grained input details that are essential for accurate image transformation. By capturing features at multiple scales, the model learns both local and global patterns, while the direct concatenation of encoder and decoder features ensures that important information is not lost during downsampling. Dilated convolutions further enhance the receptive field without increasing the number of parameters, allowing the network to incorporate broader contextual information. Similar to Model 1, the architecture follows an end-to-end learning strategy, enabling it to learn the complete transformation, including flips, in a unified manner.

## 3 Loss Function

Both models use L1Loss((Mean Absolute Error)) for training.

L1Loss is less sensitive to outliers compared to MSE (L2Loss), which is important when dealing with natural images that may have varying pixel intensities. It provides more stable gradients during training, especially for pixel-wise regression tasks and produces visually smoother results.

# 4 Early Stopping

Early stopping is implemented with the following parameters for both models:

- **Patience**: 7 epochs
- **Min Delta**: 1e-4
- **Metric**: Validation loss

Early stopping halts training when the model no longer improves on the validation set, preventing memorization of the training data. This approach saves computational resources and maintains a balance between tolerating temporary plateaus and avoiding unnecessary training, with 7 epochs being sufficient to distinguish real convergence from minor fluctuations. Additionally, only meaningful improvements reset the patience counter, preventing premature stopping caused by numerical noise.

# 5 Training Results

## 5.1 Training Configuration

Both models were trained with the following configuration:

- Optimizer: Adam with learning rate = 1e-3
- Batch size: 256
- Train/Validation split: 90%/10%
- Device: CPU (as per requirements)
- Early stopping: Patience=7, Min Delta=1e-4

## 5.2 Model 1 Training Results

Model 1 was trained for 80 epochs. The training progress is summarized below:

- **Epoch 1**: Training loss = 0.149933, Validation loss = 0.132575
- **Epoch 80**: Training loss = 0.122436, Validation loss = 0.122933

Model 1 achieves moderate loss values, prioritizing inference speed over perfect reconstruction quality. The model successfully learns the transformation while maintaining fast inference times.

## 5.3 Model 2 Training Results

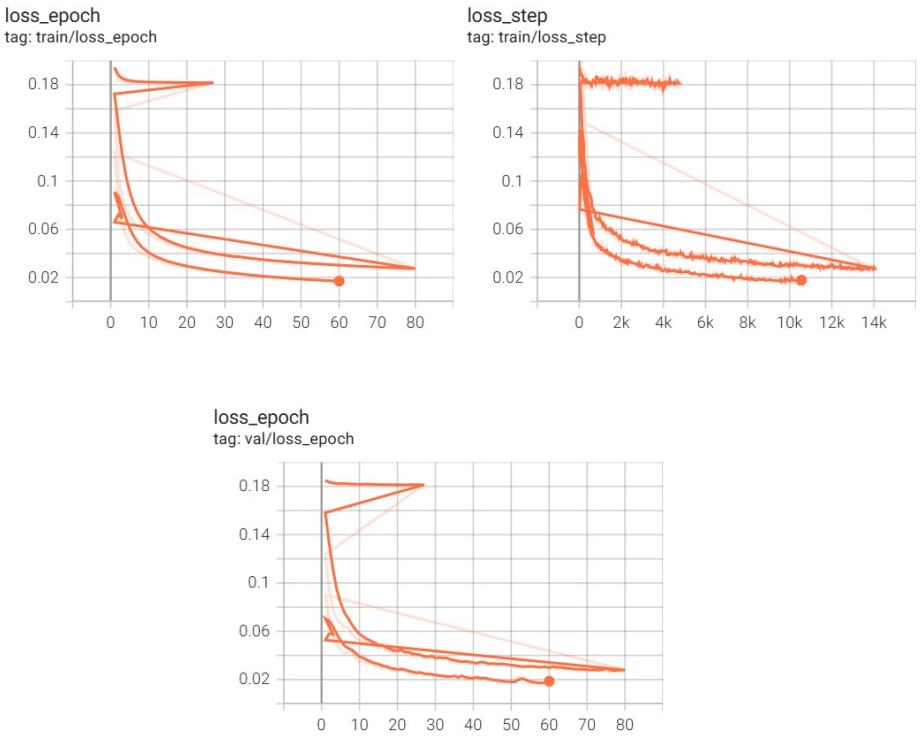
Model 2 was trained for 80 epochs. The training progress is summarized below:

- **Epoch 1**: Training loss = 0.159972, Validation loss = 0.119318
- **Epoch 80**: Training loss = 0.027211, Validation loss = 0.026560

Model 2 achieves significantly lower loss values, demonstrating superior prediction quality. The validation loss of 0.026560 indicates that the model's predictions are very close to the ground truth, though this comes at the cost of slower inference times.

## 5.4 Training Curves

The training and validation loss curves for both models are shown below. Both models were logged to TensorBoard, allowing for comparison of their training dynamics.

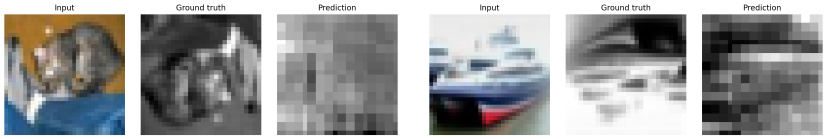



The TensorBoard logs show that Model 2 achieves much lower loss values throughout training, while Model 1 maintains higher losses but trains and infers faster.

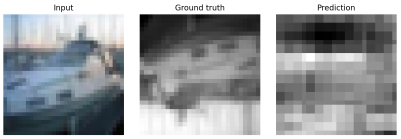
# 6 Generated Images Comparison

## 6.1 Model 1

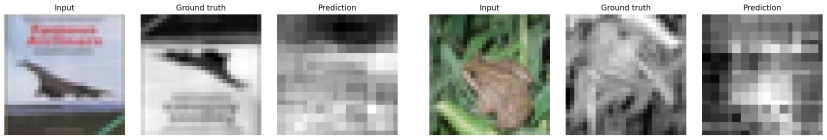
Below are 5 examples comparing the Model 1 output with the ground truth:




(a) Example 1 (b) Example 2




(c) Example 3

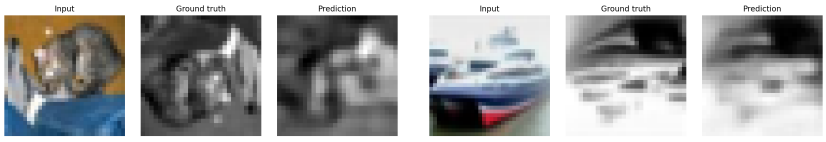



(d) Example 4 (e) Example 5

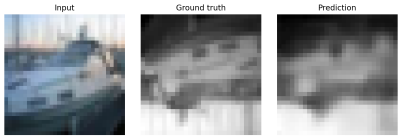
Model 1 successfully learns the RGB to grayscale conversion and handles resizing using nearest neighbor interpolation, while flip operations are learned end-to-end. Although the predictions are not perfect, they preserve the main structure and content of the images, reflecting the model's design choice to prioritize speed over perfect reconstruction, which results in higher loss values.

## 6.2 Model 2

Below are 5 examples comparing the Model 2 output with the ground truth:




(a) Example 1 (b) Example 2




(c) Example 3


(d) Example 4 (e) Example 5

Model 2 delivers high-quality predictions that closely match the ground truth, accurately learning the RGB to grayscale conversion as well as resize and flip operations. The U-Net architecture with skip connections preserves fine details, and the low validation loss is reflected in the strong visual quality, at the cost of slower inference due to prioritizing quality over speed.

# 7 Runtime Performance Benchmark

The benchmark compares:

- **Sequential CPU transforms**: Using torchvision transforms applied sequentially on CPU
- **Model inference**: Using the trained model with different batch sizes

Tests are performed on both CPU and CUDA devices with batch sizes: 1, 8, 16, 32, 64, 128, 256, 512.

## 7.1 Model 1: Runtime Performance

### 7.1.1 CPU Results

| Batch Size | Transforms CPU (s) | Model CPU (s) |
|---|---|---|
| 1 | 1.6410 | 11.6113 |
| 8 | 1.6649 | 2.6214 |
| 16 | 1.6816 | 1.6798 |
| 32 | 1.7040 | 1.7033 |
| 64 | 1.6893 | 1.5991 |
| 128 | 1.6652 | 1.6086 |
| 256 | 1.6415 | 1.5518 |
| 512 | 1.6853 | 1.9875 |

Model 1 becomes faster than sequential transforms starting from batch size 16 on CPU. The model achieves a speedup of approximately 1.06× for batch sizes 64 and 256.

### 7.1.2 CUDA Results

| Batch Size | Transforms CPU (s) | Model CUDA (s) |
|---|---|---|
| 1 | 1.6586 | 4.5851 |
| 8 | 1.6926 | 0.6388 |
| 16 | 1.7489 | 0.3852 |
| 32 | 1.7059 | 0.2258 |
| 64 | 1.7169 | 0.1469 |
| 128 | 1.7664 | 0.1049 |
| 256 | 1.7189 | 0.0904 |
| 512 | 1.7390 | 0.0840 |

Model 1 achieves significant speedups on CUDA, becoming faster than sequential transforms starting from batch size 8. The speedup increases dramatically with larger batch sizes, reaching up to 20.73× for batch size 512.

## 7.2 Model 2: Runtime Performance

### 7.2.1 CPU Results

| Batch Size | Transforms CPU (s) | Model CPU (s) |
|---|---|---|
| 1 | 1.6403 | 45.3740 |
| 8 | 1.8036 | 22.0319 |
| 16 | 1.7153 | 20.1324 |
| 32 | 1.6614 | 19.3329 |
| 64 | 1.6831 | 19.4510 |
| 128 | 1.6213 | 22.8608 |
| 256 | 1.6146 | 31.8360 |
| 512 | 1.5864 | 35.1222 |

Model 2 is significantly slower than sequential transforms on CPU across all batch sizes.

### 7.2.2 CUDA Results

| Batch Size | Transforms CPU (s) | Model CUDA (s) |
|---|---|---|
| 1 | 1.6293 | 9.0535 |
| 8 | 1.6690 | 1.3074 |
| 16 | 1.6706 | 0.7898 |
| 32 | 1.7146 | 0.6843 |
| 64 | 1.6618 | 0.5648 |
| 128 | 1.6641 | 0.4306 |
| 256 | 1.6653 | 0.2761 |
| 512 | 1.6528 | 0.1665 |

The model becomes faster than sequential CPU transforms for batch sizes 8 and above on CUDA.

## 7.3 Discussion

The results highlight a clear trade-off between speed and quality: Model 1 favors fast inference and achieves significant speedups, especially on CUDA, while Model 2 prioritizes prediction quality with lower losses but slower inference. Performance improves with larger batch sizes due to better GPU utilization, and both models benefit from GPU acceleration, with different

break-even points compared to sequential transforms depending on the device and batch size.